

ASSIGNMENT 3.1

FUNCTIONS

1.0: Lab Overview

This session introduces students to modular programming using Python functions while continuing the development of their semester-long calculator project. Students will learn how to define and call functions, pass parameters, return values, and organise programs into reusable modules. The activities gradually guide students from simple function creation to developing a modular calculator program using multiple functions.

2.0: Aim

To provide students with practical experience in designing modular Python programs using functions, parameters, return statements, and basic version control workflows.

3.0: Learning Objectives

- Define and call Python functions correctly.
- Use parameters and arguments in functions.
- Apply return statements to send results back to the program.
- Differentiate between local and global variables.
- Develop modular programs using multiple functions.

4.0: Procedure

Part 1: Creating and Calling Functions

In this exercise, you will create simple Python functions.

1. Open your IDE and create a new Python file named:

`week3_functions.py`

2. Create a function named `greet_user()` that displays a welcome message.

Example:

```
def greet_user():
```

```
    print("Welcome to COM4006")
```

3. Call the function twice.
4. Create another function called `display_course()` that prints your course name.
5. Run the program and observe the output.

Part 2: Functions with Parameters

Functions become more flexible when they accept parameters.

1. Create a function called `add(a, b)`.
2. The function should display the sum of two numbers.

Example:

```
def add(a, b):
```

```
    print(a + b)
```

3. Call the function using different values.
4. Create the following additional functions:
 - `subtract(a, b)`
 - `multiply(a, b)`
 - `divide(a, b)`
5. Test all functions using different arguments.

Part 3: Using Return Statements

In this activity, you will return values from functions.

1. Modify the `add()` function so it uses `return`.

Example:

```
def add(a, b):
```

```
    return a + b
```

2. Store the returned value in a variable.
3. Display the result using `print()`.
4. Repeat this process for:
 - subtraction
 - multiplication
 - division
5. What is the difference between `print()` and `return`?

COMPUTING PROGRAMMING (COM4006)

Part 4: Local and Global Variables

This exercise demonstrates variable scope.

1. Create a global variable named message.

Example:

```
message = "Welcome Student"
```

2. Create a function that prints the variable.
3. Create a local variable inside another function.
4. Try printing the local variable outside the function.
5. Observe and explain the error produced.

Part 5: Modular Calculator Program

Scenario-Based Exercise: Building a Modular Calculator

You have recently been hired as a junior software developer at a small technology company called **SmartCalc Solutions**.

The company wants a command-line calculator that performs:

- Addition
- Subtraction
- Multiplication
- Division

However, the senior developer has specified the following:

- Each operation must use a separate function.
- Each function must accept two parameters.
- Each function must return a value.
- The main program must handle all user interaction.

Task Requirements

Your program must:

1. Display a menu:
 - 1 Addition
 - 2 Subtraction
 - 3 Multiplication

COMPUTING PROGRAMMING (COM4006)

- 4 Division
- 2. Ask the user to enter two numbers.
- 3. Call the correct function based on the user's choice.
- 4. Display the result clearly.
- 5. Handle invalid menu choices.
- 6. Prevent division by zero.

Part 6: Challenge Task – Extended Calculator

Extend your calculator program by adding at least TWO additional operations:

Possible examples:

- Power
- Modulus
- Square root
- Percentage
- Average

Part 7: Introduction to Git and GitHub

In this activity, you will practise basic version control.

1. Open terminal or command prompt.
2. Navigate to your project folder.
3. Initialise a Git repository:

`git init`

4. Add your Python file:

`git add.`

5. Create your first commit:

`git commit -m "Added modular calculator"`

6. Create a GitHub repository and upload your project.

COMPUTING PROGRAMMING (COM4006)**Part 8: IDE Debugging Practice**

Using Visual Studio Code:

1. Run your calculator program.
2. Set breakpoints.
3. Debug one of your functions.
4. Observe how variables change during execution.

5.0: Student Reflection Questions

- Why are functions important in programming?
- What are the advantages of modular programming?
- What is the difference between local and global variables?
- Why is the return statement useful?
- What is the purpose of Git in programming projects?
- How can modular programs improve teamwork?

6.0: Conclusion

In this laboratory session, students practised designing modular Python programs using functions, parameters, return statements, and variable scope. Students also gained introductory experience with Git, GitHub, and debugging tools while extending their calculator project into a more structured and reusable program.